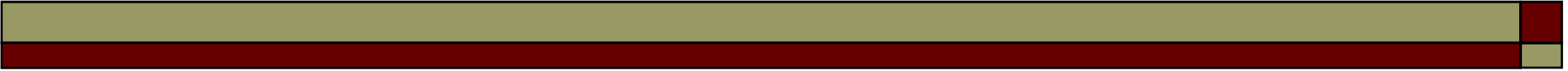
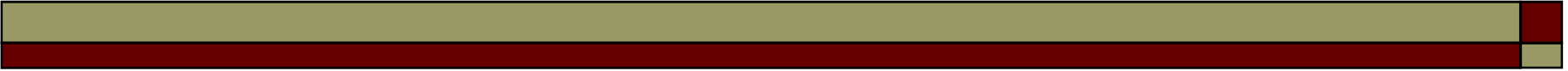


Simulated Annealing Search Algorithm



Local Search algorithms

- Search algorithms like breadth-first, depth-first or A* explore all the search space systematically by keeping one or more paths in memory and by recording which alternatives have been explored.
- When a goal is found, the path to that goal constitutes a solution.
- Local search algorithms can be very helpful if we are interested in the solution state but not in the path to that goal. They operate only on the current state and move into neighboring states .

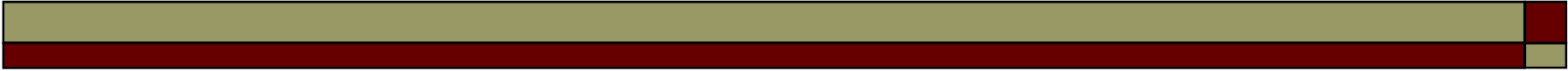


Local Search algorithms

- Local search algorithms have 2 key advantages:
 - They use very little memory
 - They can find reasonable solutions in large or infinite (continuous) state spaces.
- Some examples of local search algorithms are:
 - Hill-climbing
 - Simulated annealing
 - Genetic Algorithms

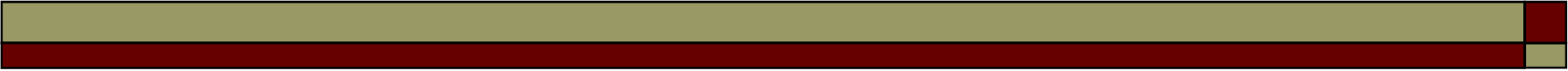
Annealing

- Annealing is a thermal process for obtaining low energy states of a solid in a heat bath.
- The process contains two steps:
 - Increase the temperature of the heat bath to a maximum value at which the solid melts.
 - Decrease carefully the temperature of the heat bath until the particles arrange themselves in the ground state of the solid. Ground state is a minimum energy state of the solid.
- The ground state of the solid is obtained only if the maximum temperature is high enough and the cooling is done slowly.



Simulated Annealing

- We can apply this algorithm to generate a solution to combinatorial optimization problems assuming an analogy between them and physical many-particle systems with the following equivalences:
 - Solutions in the problem are equivalent to states in a physical system.
 - The cost of a solution is equivalent to the “energy” of a state.



Simulated Annealing

- To apply simulated annealing with optimization purposes we require the following:
 - A successor function that returns a “close” neighboring solution given the actual one. This will work as the “disturbance” for the particles of the system.
 - A target function to optimize that depends on the current state of the system. This function will work as the energy of the system.
- The search is started with a randomized state. In a polling loop we will move to neighboring states always accepting the moves that decrease the energy while only accepting bad moves accordingly to a probability distribution dependent on the “temperature” of the system.

Simulated Annealing

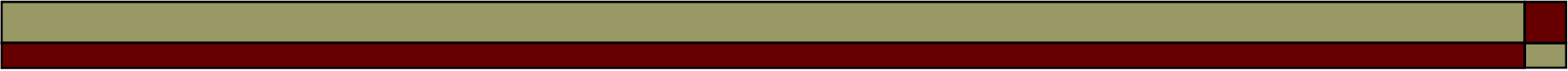
□ Decrease the temperature slowly, accepting less bad moves at each temperature level until at very low temperatures the algorithm becomes a greedy hill-climbing algorithm.

- The distribution used to decide if we accept a bad movement is known as Boltzmann distribution.

$$P(\gamma) = \frac{e^{-E_\gamma/T}}{Z(T)},$$

$$Z(T) = \sum_{\gamma'} e^{-E_{\gamma'}/T},$$

- This distribution is very well known in solid physics and plays a central role in simulated annealing. Where γ is the current configuration of the system, E_γ is the energy related with it, and Z is a normalization constant.



Here's a simple algorithm for simulated annealing search in plain statements:

- Choose an initial solution.
- Set an initial temperature.
- While the temperature is above a minimum value:
 - Generate a neighboring solution.
 - Calculate the change in cost between the current and new solution.
 - If the new solution is better, accept it.
 - If the new solution is worse, accept it with a probability based on:
 - The current temperature
 - How much worse the new solution is e. Update the current solution if accepted.
 - f. Decrease the temperature according to a cooling schedule.
- Return the best solution found.

Simulated annealing

function SIMULATED-ANNEALING(*problem*, *schedule*) **return** a solution state

input: *problem*, a problem

schedule, a mapping from time to temperature

local variables: *current*, a node.

next, a node.

T, a “temperature” controlling the prob. of downward steps

current $\leftarrow$ MAKE-NODE(INITIAL-STATE[*problem*])

for *t* $\leftarrow$ 1 **to** ∞ **do**

T $\leftarrow$ *schedule*[*t*]

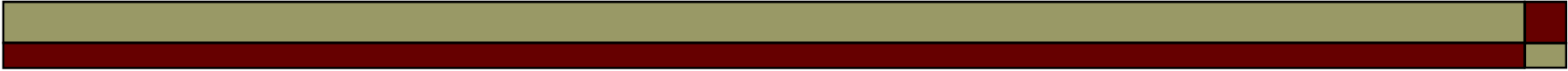
if *T* = 0 **then return** *current*

next $\leftarrow$ a randomly selected successor of *current*

$\Delta E \leftarrow$ VALUE[*next*] - VALUE[*current*]

if $\Delta E > 0$ **then** *current* $\leftarrow$ *next*

else *current* $\leftarrow$ *next* only with probability $e^{\Delta E / T}$



Temperature T

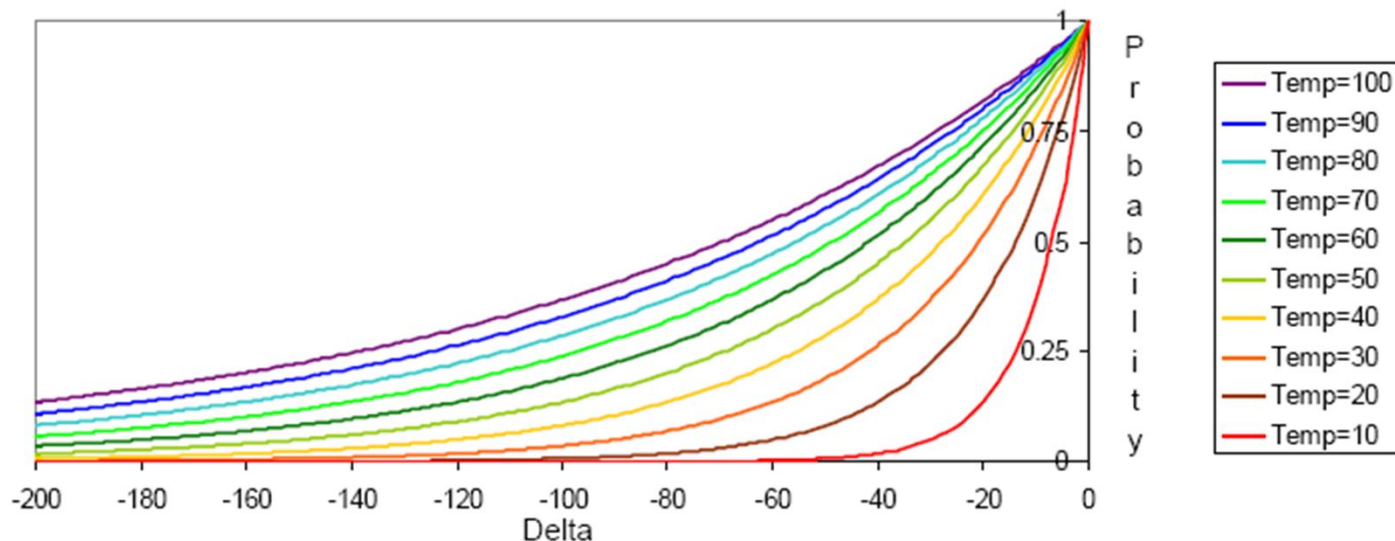
- high T : probability of “locally bad” move is higher
- low T : probability of “locally bad” move is lower
- typically, T is decreased as the algorithm runs longer
- i.e., there is a “temperature schedule”

Simulated Annealing

□ Acceptance criterion and cooling schedule

if ($\Delta \geq 0$) accept

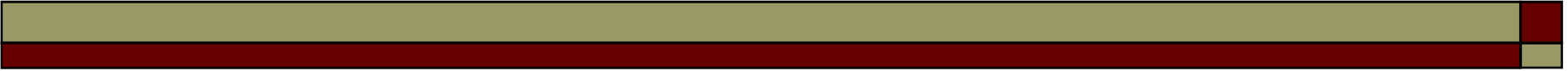
else if ($random < e^{\Delta / Temp}$) accept, else reject /* $0 \leq random \leq 1$ */



Initially temperature is very high (most bad moves accepted)

Temp slowly goes to 0, with multiple moves attempted at each temperature

Final runs with temp=0 (always reject bad moves) greedily “quench” the system



Applications

- Basic Problems
 - Traveling salesman
 - Graph partitioning
 - Matching problems
 - Graph coloring
 - Scheduling
- Engineering
 - VLSI design
 - Placement
 - Routing
 - Array logic minimization
 - Layout
 - Facilities layout
 - Image processing
 - Code design in information theory